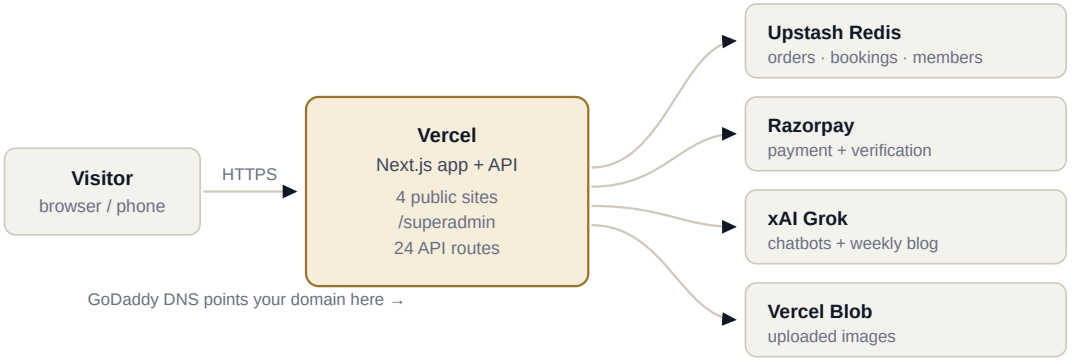
TNWLA · STAND FIRM · JENI · HARMONY

Launch Runbook

Everything between the code on your desk and a working site on your domain, in the order it has to happen. Nine phases. Two of them will lose your data if you skip them, and they are marked. Work top to bottom — later phases assume earlier ones are done.



One Next.js application serves all four brands and the admin portal. It is not four sites — `/`, `/stand-firm`, `/jeni`, `/harmonic` and `/superadmin` are routes in the same deploy, so one push updates everything. Only Upstash is required; the other three degrade gracefully when their keys are absent.

00 Build it once on your machine DO NOT SKIP

BEFORE YOU TOUCH GITHUB · ~10 MINUTES

THIS CODE HAS NEVER BEEN COMPILED

The package registry was blocked in the environment I built it in, so I could never run `npm install` or `next build`. I verified the logic by executing it in isolation — 79 assertions, all passing — and checked every import, export and route statically. But a real TypeScript compile has not happened. Do it now, on your machine, where a mistake costs a minute instead of a failed deploy.

```
cd "Stand-Firm-Legal-main"

npm install
npm run build
```

Expect it to take a couple of minutes — the build also fetches legal news as a pre-step. If TypeScript reports errors, paste them to me and I will fix them. Do not push until this command finishes cleanly.

Then run it locally and click around

```
npm run dev
# open http://localhost:3000
```

Check the home page, `/stand-firm`, `/jeni`, `/harmonic`, `/membership`, `/books`, `/events` and `/superadmin`. Locally the data goes to a file at `.data/db.json`, so everything works without any accounts set up yet.

01 Get the code into GitHub DO NOW

~5 MINUTES

YOUR FOLDER IS NOT A CLONE

`Stand-Firm-Legal-main` is the name GitHub gives a downloaded ZIP. There is no `.git` directory in it, so `git push` will not work from here. Pick one of the two routes below.

Route A — you have the repo cloned somewhere else RECOMMENDED

Copy the working files over your clone, then commit there. This keeps your history intact.

```
# copy everything except the local data and build output
robocopy "Stand-Firm-Legal-main" "path\to\your\clone" /E /XD node_modules .next .data .git

cd "path\to\your\clone"
git add -A
git commit -m "Four-brand platform: server build, events, superadmin, AI"
git push origin main
```

Route B — start fresh from this folder

```
cd "Stand-Firm-Legal-main"

git init
git add -A
git commit -m "Four-brand platform"
git branch -M main
git remote add origin https://github.com/YOUR-USER/YOUR-REPO.git
git push -u origin main --force
```

CHECK BEFORE YOU PUSH

`.gitignore` already excludes `node_modules`, `.next`, `.data/` and `.env*`. Confirm `git status` does not list any of those — especially `.env.local`, which will hold real keys shortly.

02 Create the database BEFORE ANY REAL USE

IN VERCEL · ~3 MINUTES

WITHOUT THIS, EVERY ORDER IS SILENTLY LOST

Vercel's filesystem is read-only and thrown away between requests. The app falls back to a local JSON file, which works perfectly on your laptop and fails invisibly in production — a customer books a seat, sees a confirmation, and the record evaporates. Set this up *before* you send anyone to the site.

1. In your Vercel project, open the **Storage** tab.
2. Choose **Marketplace Database Providers → Upstash → Redis**. The free tier is generous and is plenty for this.
3. Name it something like `tsjh-store`, pick the region closest to Chennai (Singapore or Mumbai), and create it.
4. Connect it to your project when prompted. Vercel injects the credentials automatically as environment variables — you do not have to copy anything by hand.

EITHER NAMING WORKS

The storage layer accepts `KV_REST_API_URL` / `KV_REST_API_TOKEN` *or* `UPSTASH_REDIS_REST_URL` / `UPSTASH_REDIS_REST_TOKEN`. Whichever pair Vercel injects, it will be picked up. The server logs which driver it chose on the first write, so you can confirm it in the Vercel function logs.

This one store holds everything: orders, bookings, enquiries, members, events, feedback, blog drafts, content overrides and themes. If you ever outgrow it, only `lib/server/db.ts` changes — every caller goes through its six functions.

03 Set the environment variables DO NOW

VERCEL → SETTINGS → ENVIRONMENT VARIABLES · ~10 MINUTES

Generate the two secrets first — run these locally and copy the output:

```
# SESSION_SECRET
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"

# CRON_SECRET
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

# SUPERADMIN_PASSWORD_HASH — pick your own password
npm run hash-password -- "your new master password"
```

VARIABLE	VALUE	NEEDED?
NEXT_PUBLIC_SITE_URL	Your live domain, no trailing slash — e.g. `https://tnwla-madras.com`. Drives the sitemap, canonical tags and every share preview.	YES
SESSION_SECRET	The 96-character string from above. Signs the admin cookie.	YES
SUPERADMIN_USER	`Master - TSJH`	DEFAULT FINE
SUPERADMIN_PASSWORD_HASH	The `scrypt$…` line from `npm run hash-password`.	YES
KV_REST_API_URL KV_REST_API_TOKEN	Injected by Upstash in phase 02. Do not type these by hand.	YES
CRON_SECRET	The 64-character string from above. Lets the hourly news job authenticate.	YES
GROK_API_KEY	From console.x.ai. Powers the chatbots and the weekly blog draft.	OPTIONAL
BLOB_READ_WRITE_TOKEN	Create a Blob store under Vercel → Storage. Enables image upload in the appearance editor.	OPTIONAL
RAZORPAY_KEY_ID RAZORPAY_KEY_SECRET	See phase 07 — deliberately **not** set in Production yet.	LATER

> **WHICH ENVIRONMENTS**
>
> Tick **Production**, **Preview** and **Development** for all of them except the Razorpay pair. Vercel needs a redeploy for new variables to take effect — changing one does not update a running deployment on its own.

The three optional keys all fail softly by design. Without Grok, the chatbots use the keyword answers they have always used. Without Blob, the appearance editor still changes colours, fonts and sizes and only disables the image picker, with a line on screen saying why.

04 Deploy and check it DO NOW

~5 MINUTES

Vercel builds automatically on push. If the project is already connected to the repo, phase 01 has already triggered a deploy — open the **Deployments** tab and watch it.

> **ONE PROJECT SETTING TO VERIFY**
>
> Under **Settings → General**, the Framework Preset must be **Next.js** and the output directory must be left on the default. If this project was ever configured as a static site — with an output directory of `out` — clear that. The app is a server build now; a static output setting will produce a site where every form and the entire admin portal returns 404.

Once the deploy is green, walk these in order on the `.vercel.app` URL:

- [ ] Home page loads, hero video plays and loops.
- [ ] `/stand-firm` — practice mega-menu opens, a sub-topic page loads, no TNWLA branding anywhere.
- [ ] `/membership` — the category dropdown opens the form.
- [ ] `/books` — add a title, submit with a phone number.
- [ ] `/superadmin` — redirects you to the login screen rather than showing anything.

That last one is the important check. If `/superadmin` shows data without asking for a password, stop and tell me — it would mean the middleware is not running.

05 First sign-in to Superadmin DO NOW

~5 MINUTES

You asked how you get the link, given it was never deployed before. There is no separate deployment and no separate URL to obtain — the portal is a route inside the same application, so it went live with everything else:

```
https://your-domain.com/superadmin
```

It is excluded from `robots.txt`, carries a `noindex` tag, and every request passes through middleware that redirects anyone without a valid signed cookie. It will not appear in search results and there is no link to it anywhere on the public site — you reach it by typing the address.

Signing in

Username	`Master - TSJH` — matched loosely on case and spacing, so `master - tsjh` also works.
Password	Whatever you hashed in phase 03. If you skipped that step it is still `Enterprises@2026`.

CHANGE THE PASSWORD BEFORE ANYONE ELSE KNOWS THE URL

The default hash is committed in the source, which means it is public. Run `npm run hash-password -- "something only you know"`, paste the result into `SUPERADMIN_PASSWORD_HASH`, and redeploy. Sessions last 12 hours.

What is inside

Four brand tabs across the top; the panels available change per brand. Under TNWLA you get **Sessions** (create and run events), **Enquiries**, **Blog** (read and publish the weekly draft), **Content**, **Appearance** and **Letterhead**. Jeni and Harmony add **Orders**. Everything is designed to work on a phone as well as a desktop.

06 Point the GoDaddy domain at Vercel DO NOW

~10 MINUTES, THEN UP TO 48 HOURS TO PROPAGATE

1. In Vercel: **Settings → Domains → Add**, and enter your domain. Add both `yourdomain.com` and `www.yourdomain.com`.
2. Vercel shows you the DNS records it wants. Leave that tab open.
3. In GoDaddy: **My Products → your domain → DNS → Manage Zones**.
4. Add the records exactly as below, editing the existing `@` and `www` entries rather than creating duplicates.

TYPE	NAME	VALUE	TTL
A	@	`76.76.21.21`	1 hour
CNAME	www	`cname.vercel-dns.com`	1 hour

> **USE WHATEVER VERCEL SHOWS YOU, NOT THIS TABLE**
>
> Those values are the current standard ones, but Vercel occasionally issues project-specific targets. The panel from step 2 is authoritative — if it differs, follow it.

GoDaddy usually applies changes within an hour. Vercel provisions the HTTPS certificate on its own once it sees the records; the domain will show **Invalid Configuration** until then, which is normal. Once it goes green, set `NEXT_PUBLIC_SITE_URL` to the real domain if you have not already, and redeploy so the sitemap and share previews point at the right host.

07 Razorpay, while it is under review

YOU CAN LAUNCH WITHOUT IT

NOW, AND AGAIN ON APPROVAL

Your dashboard says the application is under review and Test Mode is on. That is exactly the state the code was written for, and the answer is better than you might expect: **you can take real money today without Razorpay at all.**

What happens with no Razorpay keys set

The checkout falls back to the UPI flow the site already had. A customer places an order, the server prices it from your own catalogue, and they pay by UPI to `grace2jeni-8@okicici` — real money, straight into your account. They type the reference, the order lands in Superadmin as **awaiting verification**, and somebody at the office clears it against the bank statement.

That is a manual step, but it is honest and it works from day one. It is also the flow the site was already using before any of this.

> **DO NOT PUT TEST KEYS IN PRODUCTION**
>
> It is tempting, because test keys work immediately. But a test key makes Razorpay Checkout accept a fake card and report success — the server would then mark a real customer's order **paid** with no money having moved. Keep Production free of Razorpay credentials until your live keys arrive.

Testing the automated flow safely

1. In Razorpay with **Test Mode** on: **Account & Settings → API Keys → Generate Test Key**. You get a `rzp_test_…` id and a secret.
2. In Vercel, add `RAZORPAY_KEY_ID` and `RAZORPAY_KEY_SECRET` ticking **Preview** and **Development** only. Leave Production unticked.
3. Open a preview deployment and place an order. Use Razorpay's test card `4111 1111 1111 1111`, any future expiry, any CVV.
4. Confirm the order shows as **paid** in Superadmin. That proves signature verification, capture checking and amount matching all work.

On the day you are approved

Switch the dashboard out of Test Mode, generate live keys, and add the same two variables to **Production**. Redeploy. Nothing in the code changes — the app detects the keys and switches from the manual UPI flow to automatic verification on its own.

> **WHAT THE SERVER CHECKS BEFORE MARKING ANYTHING PAID**
>
> The signature recomputes with your secret key; Razorpay itself confirms the payment is captured; and the captured amount equals the total the server calculated. That third check is the one usually left out — without it, a signed and captured ₹1 payment marks a ₹4,000 order paid.

08 Turn on the scheduled jobs DO NOW

ALREADY DONE IN THE CODE · ~2 MINUTES TO VERIFY

> **THIS IS WHAT WAS FAILING EVERY DEPLOYMENT**
>
> `vercel.json` asked for `"0 * * * *"` — hourly. The Hobby plan refuses any cron more frequent than once a day, and it refuses it **at deploy time**. Not a warning, not a disabled job: the whole deployment fails, so nothing ships and the last good build keeps serving. That is why the site looked frozen for weeks while commits kept landing on GitHub.

The schedule is now daily, so deployments succeed. The hourly behaviour did not go anywhere — it moved out of the scheduler and into the code. `getNews()` checks how old the stored list is, and the first reader after it passes an hour triggers the refresh on the way through. Traffic does the scheduling.

Two details keep that honest:

- **A lock.** Ten readers arriving at the same minute must not each fetch four RSS feeds. The first claims a short, self-expiring lock and refreshes; the rest are served the existing list immediately. The expiry means a crashed run cannot wedge the feed shut.
- **Failure returns stale news, never an error.** A publisher being down must not blank the page.

Both jobs still need `CRON_SECRET` to exist, and Superadmin can trigger either by hand — the Blog panel has a **Draft one now** button, and the news endpoint accepts a signed-in admin as well as the cron caller. If you later move to Pro, putting `"0 * * * *"` back is a one-line change and the in-code refresh simply stops being the one doing the work.

08b If you are also deploying on Render SILENT FAILURE

ONLY IF A RENDER SERVICE EXISTS · ~5 MINUTES

> **A FAILED RENDER DEPLOY KEEPS SERVING THE OLD SITE**
>
> This is the trap: Render does not take a broken build down. It leaves the last good one running. So a service that has been failing for weeks looks perfectly healthy from the outside — you visit the URL, the site loads, and nothing suggests that every commit since has been rejected.

The cause here is the same change that made the login and the stored orders possible: this is no longer a static export. `output: "export"` is gone from `next.config.mjs`, so **no `out` directory is produced any more**. A Render **Static Site** service publishes a directory, cannot find one, and fails — every time, for ever.

- **If the service type is Static Site:** it can never succeed again. Delete it and create a **Web Service** — Build `npm run build`, Start `npm start` — with the same environment variables as Vercel.
- **If it is already a Web Service:** open the latest deploy and read the log.
- **If you do not need two hosts:** delete the Render service. Running the same repo on two hosts means two URLs that can disagree, and checking the stale one is a very easy way to conclude that a working change did not ship.

Whichever you choose, note which URL is the real one and check that URL. Vercel and Render are independent deployments of the same repository; a green build on one says nothing about the other.

09 What is still owed before you tell people about it CONTENT, NOT CODE

ONGOING

None of these stop a deploy. All of them are things only you can supply, and each is marked in the source so you can find it.

- [ ] **The UPI QR image — no longer needed.** This used to wait on a `public/media/upi-qr.png` that never arrived, and until it did, both payment screens fell back to "type the UPI ID yourself". The code now draws the QR in the browser from the same UPI string the pay buttons use, carrying the amount and the order reference, so a scan leaves only the PIN to enter. It cannot go missing, and it cannot disagree with the buttons. Nothing to supply.

- [ ] **Member dates of birth**, for the birthday panel. Paste the roll as a spreadsheet into `POST /api/members/import` — a header row plus `membershipNo, memberName, dob, mobile, district, validUpTo`. Running it twice is safe; existing members are updated, not duplicated.

- [ ] **Real product names and prices.** Search the codebase for `TODO stock` — about 40 lines across the clothing, wholesale, export and dhoobam catalogues. The shop machinery is real; the stock list is a stand-in so it could be built and tested.

- [ ] **Festival dates for the year.** Search `TODO verify` in `config/festivals.config.ts`. Deepavali, Pongal, Ramzan and Good Friday move each year and carry explicit per-year dates — a year with no entry shows nothing rather than guessing wrong.

- [ ] **The masters' lineage.** Search `TODO history`. I left these blank on purpose — a lineage stated wrongly in print is a serious discourtesy in that tradition, and only the centre can say who taught whom.

- [ ] **Stand Firm partner names.** Still `[PH: Partner Name]` in `config/site.config.ts`. They were placeholders before the partnerships block moved to `/stand-firm/team`, and I did not invent any.

→ The short version

IF YOU ONLY READ ONE THING

1. `npm install && npm run build` locally. Send me any errors.
2. Copy the files into your clone and push.
3. Add Upstash Redis in Vercel → Storage. **This one is not optional.**
4. Set the environment variables, above all `SESSION_SECRET`, `SUPERADMIN_PASSWORD_HASH` and `NEXT_PUBLIC_SITE_URL`.
5. Redeploy, then check `/superadmin` asks for a password.
6. Point GoDaddy at Vercel with the A and CNAME records.
7. Leave Razorpay out of Production. Take UPI payments and verify by hand until your keys arrive.

> **WHERE TO FIND THE DETAIL**
>
> `DEPLOYMENT.md` covers hosting and the storage drivers. `TNWLA-CHANGES.md` lists every change across both passes, including the three bugs the tests caught. `.env.example` documents each variable and what happens when it is missing.

Four brands, one Next.js application, twenty-four API routes, twenty-four pages. Written for TNWLA — Madras · Stand Firm Legal Associates · Jeni Enterprises · Harmony Pranic Healing.